

```

#include <iostream>
using namespace std;

const int MAX = 100;
int visited[MAX];

class Queue {
    int arr[MAX];
    int front, rear;
public:
    Queue() { front = rear = -1; }
    bool isEmpty() { return front == -1; }
    bool isFull() { return (rear + 1) % MAX == front; }
    void enqueue(int val) {
        if (isFull()) { cout << "Queue is full\n"; return; }
        if (isEmpty()) front = rear = 0;
        else rear = (rear + 1) % MAX;
        arr[rear] = val;
    }
    int dequeue() {
        if (isEmpty()) return -1;
        int val = arr[front];
        if (front == rear) front = rear = -1;
        else front = (front + 1) % MAX;
        return val;
    }
};

// BFS from a single starting node
void BFS(int adj[MAX][MAX], int n, int SN) {
    Queue q;
    q.enqueue(SN);
    visited[SN] = 1;
    while (!q.isEmpty()) {
        int cur = q.dequeue();
        cout << cur + 1 << " ";
        for (int i = 0; i < n; i++) {
            if (adj[cur][i] == 1 && visited[i] != 1) {
                q.enqueue(i);
                visited[i] = 1;
            }
        }
    }
}

```

```

void inputEdgesToAdjMatrix(int adj[MAX][MAX], int n, int e) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = 0;
    cout << "Enter edges (source destination):\n";
    for (int i = 0; i < e; i++) {
        int u, v;
        cin >> u >> v;
        if (u >= 1 && u <= n && v >= 1 && v <= n &&
            u != v && adj[u - 1][v - 1] == 0) {
            adj[u - 1][v - 1] = 1;
            // Undirected? also: adj[v-1][u-1] = 1;
        } else {
            cout << "Invalid edge\n";
            i--;
        }
    }
}

int main() {
    int n, e;
    int adj[MAX][MAX];
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    inputEdgesToAdjMatrix(adj, n, e);
    int start;
    cout << "Enter starting node: ";
    cin >> start;
    for (int i = 0; i < n; i++) visited[i] = 0;
    cout << "\nBFS Traversal: ";
    BFS(adj, n, start - 1);
    // Handle disconnected components
    for (int i = 0; i < n; i++) {
        if (visited[i] != 1) {
            cout << "\n(disconnected, new BFS from " << i + 1 << ") ";
            BFS(adj, n, i);
        }
    }
    cout << endl;
    return 0;
}

```

WHEN TO USE BFS (Scenario Triggers)

- ▶ Question asks for the SHORTEST path in an unweighted graph
- ▶ Question mentions "level by level" or "minimum number of steps/hops"
- ▶ Finding shortest path in a maze / grid where every move costs the same
- ▶ Checking if a graph is bipartite (2-coloring using BFS layers)
- ▶ Finding all nodes within K steps/hops of a source
- ▶ Graph is DENSE (many edges) — class notes explicitly say prefer BFS here
- ▶ Social network "degrees of separation" style questions

BFS vs DFS — Which One?

- ▶ Need SHORTEST path / fewest edges? → BFS (guaranteed shortest in unweighted graph)
- ▶ Need to explore full depth, detect cycles, topological order? → DFS instead
- ▶ Graph is wide/dense? → BFS. Graph is deep/narrow? → DFS uses less memory
- ▶ BFS uses a QUEUE (FIFO). DFS uses a STACK (LIFO) — this is the core mechanical difference

MANUAL TRACE METHOD (How to Solve by Hand)

Use this exact process if the exam gives you a graph and asks you to trace BFS:

1. Pick the start node. Mark it visited. Enqueue it. Write it in the Output list.
2. Dequeue the front node from the queue.
3. Look at ALL its neighbours in order (usually smallest index / alphabetical first).
4. For each UNVISITED neighbour: mark visited, add to Output, enqueue it.
5. Repeat step 2-4 until the queue is empty.
6. If nodes remain unvisited, graph is disconnected — restart BFS from any unvisited node.

Keep a Queue column and a Visited row/table as you go (matches the class notes format) — write the queue's contents after each dequeue/enqueue step, this is what the exam checks.

COMMON GOTCHAS (Exam Traps)

⚠ Watch out for:

Disconnected graphs — same rule as DFS: if not all nodes visited from one start, restart BFS from an unvisited node.

Queue vs Stack mix-up — a very common exam trap is writing DFS logic (stack/LIFO) but calling it BFS. Double-check you enqueue ALL unvisited neighbours before moving to the next node, not just one.

1-indexed input, 0-indexed storage — same off-by-one risk as DFS. Node 1 the user types = index 0 internally.

BFS Tree — many exam questions ask you to DRAW the BFS tree, not just list the output order. The BFS tree shows parent-child edges only (the edge that FIRST discovered each node), not all original graph edges.

"Visited" marked at enqueue time, not dequeue time — mark visited the moment you enqueue a node, otherwise the same node can be added to the queue multiple times.

EXAMPLE 1 — Small Connected Graph (Trace by Hand)

Graph (undirected): A-B, A-C, B-D, B-E, C-F, E-F

Start node: A

Step-by-step trace:

- ▶ Queue: [A] → Visited: {A}
- ▶ Dequeue A → neighbours B, C (both unvisited) → enqueue both
- ▶ Queue: [B, C] → Visited: {A, B, C}
- ▶ Dequeue B → neighbours A(visited), D, E → enqueue D, E
- ▶ Queue: [C, D, E] → Visited: {A, B, C, D, E}
- ▶ Dequeue C → neighbours A(visited), F → enqueue F
- ▶ Queue: [D, E, F] → Visited: {A, B, C, D, E, F}
- ▶ Dequeue D → no unvisited neighbours
- ▶ Dequeue E → neighbours B(visited), F(visited) → nothing new
- ▶ Dequeue F → neighbours C(visited), E(visited) → nothing new
- ▶ Queue empty → done

Output order: A, B, C, D, E, F

BFS Tree (parent-child edges only): A→B, A→C, B→D, B→E, C→F

EXAMPLE 2 — Shortest Path Scenario (CP-style)

Scenario: "Find the minimum number of bus transfers to travel from Station A to Station E, given a network of direct bus routes between stations."

How to apply BFS: treat each station as a node, each direct route as an unweighted edge. Run BFS from A. The FIRST time you reach E, the number of steps taken (BFS layer/depth) is the minimum number of transfers. Track this using a dist[] array where dist[start]=0 and dist[neighbour]=dist[current]+1.

EXAMPLE 3 — Disconnected Graph (Restart Scenario)

Graph: A-B, C-D (two separate components)

Start node: A → BFS visits only A, B. Nodes C, D remain unvisited.

Exam expects you to say: "Since C is unvisited after the first BFS call, restart BFS from C." Final Output: A, B, C, D (two separate BFS calls combined).

EXAMPLE 4 — Bipartite Check Twist (Medium/Hard style)

Scenario: "A teacher wants to split students into two debate teams such that no two friends are on the same team."

How to apply BFS: model students as nodes, friendships as edges. Run BFS, coloring each node with an alternating color (0/1) from its parent. If you ever find an edge connecting two same-colored nodes, it's impossible (graph is not bipartite) — meaning the friend groups cannot be split into two teams cleanly.

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER

Before you finalize any BFS answer, check:

- Did I mark nodes visited at ENQUEUE time, not dequeue time?
- Did I process ALL neighbours of a node before moving to the next queue item?
- Did I handle disconnected components with a restart?
- If asked for BFS Tree, did I only draw parent-child (discovery) edges, not all graph edges?
- Did I follow the correct neighbour order (usually smallest index / alphabetical first)?

```

#include <iostream>
using namespace std;

const int MAX = 100;
int visited[MAX];
int startTime[MAX];
int finishTime[MAX];
int time_counter = 0;

class Stack {
    int arr[MAX];
    int top;
public:
    Stack() { top = -1; }
    bool isEmpty() { return top == -1; }
    bool isFull() { return top == MAX - 1; }
    void push(int val) {
        if (isFull()) { cout << "Stack is full\n"; return; }
        arr[++top] = val;
    }
    int pop() {
        if (isEmpty()) return -1;
        return arr[top--];
    }
    int peek() {
        if (isEmpty()) return -1;
        return arr[top];
    }
};

// Iterative DFS, records start/finish time
void DFS(int adj[MAX][MAX], int n, int SN) {
    int nextChild[MAX] = {0};
    Stack st;

    st.push(SN);
    visited[SN] = 1;
    startTime[SN] = ++time_counter;
    cout << SN + 1 << " ";
    nextChild[SN] = 0;

    while (!st.isEmpty()) {
        int cur = st.peek();
        bool found = false;

        while (nextChild[cur] < n) {
            int next = nextChild[cur];
            nextChild[cur]++;

            if (adj[cur][next] == 1 && visited[next] != 1) {
                st.push(next);
                visited[next] = 1;
                startTime[next] = ++time_counter;
                cout << next + 1 << " ";
                nextChild[next] = 0;
                found = true;
                break;
            }
        }
    }
}

```

```

        if (!found) {
            st.pop();
            finishTime[cur] = ++time_counter;
        }
    }
}

void inputEdgesToAdjMatrix(int adj[MAX][MAX], int n, int e) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = 0;

    cout << "Enter edges (source destination):\n";
    for (int i = 0; i < e; i++) {
        int u, v;
        cin >> u >> v;
        if (u >= 1 && u <= n && v >= 1 && v <= n &&
            u != v && adj[u - 1][v - 1] == 0) {
            adj[u - 1][v - 1] = 1;
            // Undirected? also: adj[v-1][u-1] = 1;
        } else {
            cout << "Invalid edge\n";
            i--;
        }
    }
}

int main() {
    int n, e;
    int adj[MAX][MAX];

    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;

    inputEdgesToAdjMatrix(adj, n, e);

    int start;
    cout << "Enter starting node: ";
    cin >> start;

    for (int i = 0; i < n; i++) visited[i] = 0;
    time_counter = 0;

    cout << "\nDFS Traversal: ";
    DFS(adj, n, start - 1);
    // Handle disconnected components
    for (int i = 0; i < n; i++) {
        if (visited[i] != 1) {
            cout << "\n(disconnected, new DFS from " << i + 1 << ") ";
            DFS(adj, n, i);
        }
    }

    cout << "\n\nNode\tStart\tFinish\n";
    for (int i = 0; i < n; i++)
        cout << i + 1 << "\t" << startTime[i] << "\t" << finishTime[i] << "\n";

    return 0;
}

```

WHEN TO USE DFS (Scenario Triggers)

- ▶ Question mentions: exploring "as deep as possible" before backtracking
- ▶ Detecting a cycle in a graph (directed or undirected)
- ▶ Checking connectivity / finding connected components
- ▶ Topological sorting (DFS-based version)
- ▶ Finding articulation points, bridges, or strongly connected components
- ▶ Maze/pathfinding where you need ANY path, not the shortest one
- ▶ Question says "go deep first", "backtrack", or gives a stack-based hint

DFS vs BFS — Which One?

- ▶ Need SHORTEST path (unweighted graph)? → use BFS instead
- ▶ Need to explore full depth / detect cycles / topological order? → DFS
- ▶ Graph is very deep and narrow? → DFS uses less memory (stack vs queue)
- ▶ Graph is wide and shallow, dense? → notes say prefer BFS for dense graphs

MANUAL TRACE METHOD (How to Solve by Hand)

Use this exact process if the exam gives you a graph and asks you to trace DFS:

1. Pick the start node. Mark it visited. Write it in the Output list.
2. Look at the start node's neighbours in order (usually alphabetical / smallest index first).
3. Go to the FIRST unvisited neighbour. Mark visited, add to Output, push to stack (mentally).
4. Repeat step 2-3 from this new node — always go DEEPER first.
5. If a node has NO unvisited neighbours left, backtrack (pop) to the previous node and try its next neighbour.
6. Stop when the stack is empty. If nodes remain unvisited, graph is disconnected — restart DFS from any unvisited node.

Track Start/Finish time: every time you VISIT a node, write down a running counter (Start). Every time you POP/backtrack from a node (no more neighbours), write the counter again (Finish).

COMMON GOTCHAS (Exam Traps)**⚠ Watch out for:**

Disconnected graphs — if not all nodes get visited from one start, you MUST restart DFS from an unvisited node. Don't just stop.

Directed vs Undirected — code only sets $\text{adj}[u][v]=1$ by default (directed). For undirected graphs, you must also set $\text{adj}[v][u]=1$.

1-indexed input, 0-indexed storage — teacher's code takes node 1,2,3... from user but stores as 0,1,2... internally. Easy to make an off-by-one mistake.

Edge classification (Tree/Forward/Back/Cross) — Back edge = pointing to an ancestor still on the stack (means a CYCLE in directed graphs). This is a favorite scenario-question trap.

Start/Finish time table — a common question format: "give the S/F time for each node" — practice this by hand, not just running code mentally.

EXAMPLE 1 — Small Connected Graph (Trace by Hand)

Graph (undirected): A-B, A-C, B-D, B-E, C-F, E-F

Start node: A

Step-by-step trace:

- ▶ Visit A (Start=1) → neighbours: B, C
- ▶ Visit B (Start=2) → neighbours: A(visited), D, E
- ▶ Visit D (Start=3) → no unvisited neighbours → Finish D=4
- ▶ Back to B → visit E (Start=5) → neighbours: B(visited), F
- ▶ Visit F (Start=6) → neighbours: C, E(visited)
- ▶ Visit C (Start=7) → neighbours: A(visited), F(visited) → Finish C=8
- ▶ Back to F → no more → Finish F=9
- ▶ Back to E → no more → Finish E=10
- ▶ Back to B → no more → Finish B=11
- ▶ Back to A → no more → Finish A=12

Output order: A, B, D, E, F, C

EXAMPLE 2 — Disconnected Graph (Restart Scenario)

Graph: A-B, C-D (two separate components)

Start node: A → DFS visits only A, B. Nodes C, D remain unvisited.

Exam expects you to say: "Since C is unvisited after the first DFS call, restart DFS from C." Final Output: A, B, C, D (two separate DFS calls combined).

EXAMPLE 3 — Cycle Detection Scenario (CP-style twist)

Scenario: "A network admin wants to know if adding a new cable between Router X and Router Y creates a loop in the network."

How to apply DFS here: run DFS from any node. If you reach a node that is already visited AND is currently an ancestor on the current path (still on stack) — that's a cycle. This maps directly to the "Back Edge" concept from class notes.

EXAMPLE 4 — Directed Graph Twist (Medium/Hard style)

Graph (directed): $A \rightarrow F$, $F \rightarrow G$, $G \rightarrow H$, and also $A \rightarrow F$, $F \rightarrow G$ directly again shown from notes (two paths reaching H)

Key exam question pattern: "Is there an edge classification issue here?" — if a path revisits a node already fully finished (Finish time set) via a different route, that's a FORWARD or CROSS edge, not a cycle. Only back-to-an-ancestor-on-stack counts as a cycle (Back Edge).

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER**Before you finalize any DFS answer, check:**

- Did I mark start/finish times correctly (only marks when visiting / when fully backtracking)?
- Did I handle disconnected components with a restart?
- Directed graph — did I check for back edges (cycle) correctly, not just "already visited"?
- Did I follow the correct neighbour order (usually smallest index / alphabetical first)?

PRIM'S ALGORITHM (MST) • SIDE 1: CODE

```
#include <iostream>
using namespace std;

const int MAX = 100;
const int INF = 99999999;

void inputEdgesToAdjMatrix(int adj[MAX][MAX], int n, int e) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = (i == j) ? 0 : INF;
    cout << "Enter edges (Source Destination Weight):\n";
    for (int i = 0; i < e; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        if (u >= 1 && u <= n && v >= 1 && v <= n &&
            u != v && adj[u - 1][v - 1] == INF) {
            adj[u - 1][v - 1] = w;
            adj[v - 1][u - 1] = w; // undirected: both directions
        } else {
            cout << "Invalid edge. Try again.\n";
            i--;
        }
    }
}

// Prim's Algorithm - builds MST starting from a given node
void primMST(int adj[MAX][MAX], int n, int startNode) {
    int key[MAX]; // min edge weight to connect node into MST
    int parent[MAX]; // stores MST tree structure
    bool inMST[MAX]; // true if node already included
    for (int i = 0; i < n; i++) {
        key[i] = INF;
        inMST[i] = false;
        parent[i] = -1;
    }
    key[startNode] = 0;
    for (int count = 0; count < n; count++) {
        // pick min-key node not yet in MST
        int u = -1, minVal = INF;
        for (int i = 0; i < n; i++) {
            if (!inMST[i] && key[i] < minVal) {
                minVal = key[i];
                u = i;
            }
        }
        if (u == -1) break; // remaining nodes unreachable
        inMST[u] = true;
        // update keys of adjacent nodes
        for (int v = 0; v < n; v++) {
            if (adj[u][v] != INF && !inMST[v] && adj[u][v] < key[v]) {
                key[v] = adj[u][v];
                parent[v] = u;
            }
        }
    }
}
```

```
        cout << "\nEdge \tWeight\n";
        int totalWeight = 0;
        for (int i = 0; i < n; i++) {
            if (parent[i] != -1) {
                cout << parent[i]+1 << " - " << i+1 << "\t" << key[i] << "\n";
                totalWeight += key[i];
            }
        }
        for (int i = 0; i < n; i++)
            if (!inMST[i])
                cout << "Node " << i+1 << " is unreachable.\n";
        cout << "\nTotal MST Weight = " << totalWeight << endl;
    }
}

int main() {
    int n, e;
    int adj[MAX][MAX];
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    inputEdgesToAdjMatrix(adj, n, e);
    int start;
    cout << "\nEnter starting node: ";
    cin >> start;
    primMST(adj, n, start - 1);
    return 0;
}
```

WHEN TO USE PRIM'S (Scenario Triggers)

- ▶ Question asks for Minimum Spanning Tree (MST) — connect all nodes at minimum total cost
- ▶ Graph is DENSE (many edges, close to complete graph) — class notes: "Prim's — node [Dense Graph]"
- ▶ "Connect all cities/offices/computers with minimum total cable/road cost" style scenarios
- ▶ You already know the graph is CONNECTED (Prim's naturally grows one tree from a start node)
- ▶ Question gives weighted, undirected edges and asks to minimize total connection cost

Prim's vs Kruskal's — Which One?

- ▶ Graph is DENSE → Prim's is more efficient (grows tree node-by-node)
- ▶ Graph is SPARSE or edges given as a simple list → Kruskal's (sorts edges, uses Union-Find)
- ▶ Need to pick edges in strict sorted order? → Kruskal's
- ▶ Both ALWAYS give the same total MST weight — just build it differently
- ▶ Prim's grows ONE connected tree outward. Kruskal's can grow MULTIPLE separate pieces that merge later

MANUAL TRACE METHOD (Selected-Node Table, from Class Notes)

Use this exact process — it matches the "Selected node" table format from class:

1. Make a table: rows = selected nodes so far, columns = ALL nodes in the graph.
2. Start: mark the start node's row with 0 for itself, INF (α) for all others.
3. Pick the SMALLEST unvisited value in the current row → that becomes the next selected node.
4. New row: copy previous row, but UPDATE any column where a smaller edge weight exists from the newly selected node.
5. Circle the value you just picked (this becomes part of the MST).
6. Repeat until every node has been selected (circled) once.
7. Sum all the circled (selected) values = Total MST weight.

COMMON GOTCHAS (Exam Traps)

⚠ Watch out for:

Undirected requirement — Prim's REQUIRES $\text{adj}[u][v] = \text{adj}[v][u]$ (both directions set). Forgetting this breaks the algorithm silently.

Disconnected graph — if some node's $\text{key}[]$ stays INF forever, it's unreachable. MST is only valid for connected graphs; say so explicitly if asked.

MST is NOT unique if there are equal-weight edges — multiple valid MSTs can exist, but total weight is always the same.

Don't confuse 'key[]' (cheapest edge INTO the MST) with 'shortest path from source' (that's Dijkstra) — Prim's minimizes EDGE weight, not cumulative distance.

n nodes → MST always has exactly $n-1$ edges. If your edge count doesn't match, something's wrong.

EXAMPLE 1 — Small Graph (Trace by Hand)

Graph (undirected, weighted): A-B=2, A-C=3, B-C=1, B-D=4, C-D=5

Start node: A

Step-by-step trace:

- ▶ $\text{key}[A]=0$, $\text{key}[B]=\text{INF}$, $\text{key}[C]=\text{INF}$, $\text{key}[D]=\text{INF}$
- ▶ Select A ($\text{key}=0$) → update: $\text{key}[B]=2$ (A-B), $\text{key}[C]=3$ (A-C)
- ▶ Select B ($\text{key}=2$, smallest) → update: $\text{key}[C]=\min(3,1)=1$ (B-C better), $\text{key}[D]=4$ (B-D)
- ▶ Select C ($\text{key}=1$, smallest) → update: $\text{key}[D]=\min(4,5)=4$ (no change, B-D still better)
- ▶ Select D ($\text{key}=4$) → done, all nodes selected

MST Edges: A-B(2), B-C(1), B-D(4) → Total Weight = 7

EXAMPLE 2 — Disconnected Graph

Graph: A-B=5, C-D=3 (two separate components)

Start from A → only A, B get included (key stays INF for C, D forever).

Exam expects you to say: "No MST exists for the full graph since it's disconnected. Node C and D are unreachable from A." Prim's cannot fix this by restarting (unlike BFS/DFS) — it only builds ONE tree from ONE component.

EXAMPLE 3 — Real-World Scenario (CP-style)

Scenario: "An ISP wants to lay fiber cable connecting 6 towns with minimum total cable length. Cable cost between any two towns is given."

How to apply Prim's: towns = nodes, possible cable routes = weighted edges (cost = length/price). Since this is likely a DENSE graph (cost given between most/all town pairs), Prim's is the efficient choice over Kruskal's. Start from any town, grow the MST — the final total weight is the minimum total cable cost.

EXAMPLE 4 — Tie-Breaking Twist (Medium/Hard style)

Scenario: two different edges have the EXACT same weight during selection (e.g., $\text{key}[C]=4$ and $\text{key}[D]=4$ at the same step).

Correct handling: either choice is valid — both lead to a valid MST with the same TOTAL weight, though the specific edges included may differ. If the exam asks "is the MST unique?", the answer is NO when tie-breaking edges exist — always check for equal-weight edges before claiming uniqueness.

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER

Before you finalize any Prim's answer, check:

- Did I treat the graph as undirected (set both $\text{adj}[u][v]$ and $\text{adj}[v][u]$)?
- Did I pick the SMALLEST key value at each step, not the smallest node index?
- Does my final edge count equal $n-1$ (n = number of nodes)?
- If the graph is disconnected, did I state that no full MST exists?
- Did I check for tied edge weights before claiming the MST is unique?

KRUSKAL'S ALGORITHM (MST) • SIDE 1: CODE

```
#include <iostream>
using namespace std;
const int MAX = 100;
const int MAXE = MAX * MAX;

int parent[MAX]; // manual Union-Find structure

// find the "group leader" of a node (with path compression)
int findParent(int node) {
    if (parent[node] == node) return node;
    parent[node] = findParent(parent[node]);
    return parent[node];
}

// join two groups together
void unionNodes(int a, int b) {
    int pa = findParent(a);
    int pb = findParent(b);
    if (pa != pb) parent[pa] = pb;
}

struct Edge {
    int u, v, weight;
};

// simple manual sort (ascending by weight)
void sortEdges(Edge edges[], int e) {
    for (int i = 0; i < e - 1; i++) {
        for (int j = 0; j < e - i - 1; j++) {
            if (edges[j].weight > edges[j + 1].weight) {
                Edge temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }
}

void inputEdges(Edge edges[], int n, int e) {
    cout << "Enter edges (Source Destination Weight):\n";
    for (int i = 0; i < e; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        if (u >= 1 && u <= n && v >= 1 && v <= n && u != v) {
            edges[i].u = u - 1;
            edges[i].v = v - 1;
            edges[i].weight = w;
        } else {
            cout << "Invalid edge. Try again.\n";
            i--;
        }
    }
}
```

```
// Kruskal's Algorithm - picks smallest edges that don't form a cycle
void kruskalMST(Edge edges[], int n, int e) {
    sortEdges(edges, e);
    for (int i = 0; i < n; i++) parent[i] = i; // each node its own leader
    int edgeCount = 0, totalWeight = 0;
    cout << "\nEdge \tWeight\n";
    for (int i = 0; i < e && edgeCount < n - 1; i++) {
        int u = edges[i].u, v = edges[i].v;
        if (findParent(u) != findParent(v)) { // no cycle formed
            unionNodes(u, v);
            cout << u+1 << " - " << v+1 << "\t" << edges[i].weight << "\n";
            totalWeight += edges[i].weight;
            edgeCount++;
        }
    }
    if (edgeCount < n - 1) {
        cout << "\nGraph is disconnected. MST incomplete.\n";
        cout << "Edges used: " << edgeCount << " / " << n - 1 << "\n";
    }
    cout << "\nTotal MST Weight = " << totalWeight << endl;
}

int main() {
    int n, e;
    Edge edges[MAXE];
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    inputEdges(edges, n, e);
    kruskalMST(edges, n, e);
    return 0;
}
```

KRUSKAL'S ALGORITHM (MST) • SIDE 2: WHEN, HOW & GOTCHAS

WHEN TO USE KRUSKAL'S (Scenario Triggers)

- ▶ Question asks for Minimum Spanning Tree (MST) — same goal as Prim's, different method
- ▶ Graph is SPARSE (relatively few edges) or given directly as an EDGE LIST
- ▶ You need to process edges in strictly sorted (increasing weight) order
- ▶ Graph may NOT be fully connected initially — Kruskal's naturally works with edge lists from disjoint pieces
- ▶ Question hints at "union", "grouping", "merging components", or "avoid cycles"

Kruskal's vs Prim's — Which One?

- ▶ Graph is SPARSE / edges given as list → Kruskal's (sort + Union-Find)
- ▶ Graph is DENSE (near-complete) → Prim's is more efficient
- ▶ Kruskal's builds MULTIPLE tree fragments that merge together over time
- ▶ Prim's builds ONE single tree that grows outward from a start node
- ▶ Both always produce the same TOTAL MST weight for the same graph

MANUAL TRACE METHOD (Node-by-Node Edge List, from Class Notes)

Use this exact process — it matches the class notes' per-node edge listing format:

1. List ALL edges sorted by weight (ascending), OR group them by each node as shown in notes.
2. Go through edges in increasing weight order, one at a time.
3. For each edge (u,v): check if u and v are ALREADY in the same group (same 'root'/leader).
4. If NOT in the same group → SELECT this edge (box it), then UNION the two groups together.
5. If ALREADY in the same group → REJECT this edge (cross it out with X) — it would form a cycle.
6. Stop once you've selected exactly (n-1) edges, where n = total number of nodes.
7. Sum all selected (boxed) edge weights = Total MST weight.

COMMON GOTCHAS (Exam Traps)

⚠ Watch out for:

Sorting is MANDATORY — Kruskal's only works correctly if edges are processed in increasing weight order. Skipping this gives a wrong (non-minimum) tree.

Cycle check is the CORE logic — use Union-Find (findParent) to check 'same group' in O(near-1) time, not a slow manual cycle search.

Path compression matters — `parent[node] = findParent(parent[node])` speeds up future lookups; without it, worst-case behavior degrades badly on large graphs.

Disconnected graph — Kruskal's will simply select fewer than (n-1) edges and stop; you must explicitly report the graph is disconnected.

Equal-weight edges can be processed in ANY order among themselves — this may produce a different (but equally valid) MST than Prim's picks.

KRUSKAL'S ALGORITHM (MST) • SIDE 3: WORKED EXAMPLES

EXAMPLE 1 — Small Graph (Trace by Hand)

Graph (undirected, weighted): A-B=2, A-C=3, B-C=1, B-D=4, C-D=5

Sorted edges: B-C(1), A-B(2), A-C(3), B-D(4), C-D(5)

Step-by-step trace:

- ▶ B-C (1): B and C in different groups → SELECT. Union{B,C}.
- ▶ A-B (2): A and B in different groups → SELECT. Union{A,B,C}.
- ▶ A-C (3): A and C now in SAME group → REJECT (would form a cycle).
- ▶ B-D (4): B and D in different groups → SELECT. Union{A,B,C,D}.
- ▶ 3 edges selected = n-1 (n=4) → STOP, MST complete.

MST Edges: B-C(1), A-B(2), B-D(4) → Total Weight = 7

Note: Same total weight as Prim's example on the same graph — confirms both algorithms agree, even though the exact edges picked can differ.

EXAMPLE 2 — Disconnected Graph

Graph: A-B=5, C-D=3 (two separate components)

Sorted: C-D(3), A-B(5). Process both — SELECT C-D, SELECT A-B.

Only 2 edges selected, but n=4 needs n-1=3 edges. Exam expects you to say: "MST incomplete — the graph is disconnected. Only individual spanning trees exist for each component (A-B and C-D separately), no single MST connects all 4 nodes."

EXAMPLE 3 — Real-World Scenario (CP-style)

Scenario: "A telecom company has a list of possible fiber links between towns, each with an installation cost. They want to connect all towns at minimum total cost, and only a SPARSE set of possible links exists."

How to apply Kruskal's: since only a sparse/limited list of possible links exists (not all town pairs), Kruskal's is the natural fit — sort the given links by cost, use Union-Find to avoid forming loops (redundant connections), and stop once all towns are connected.

EXAMPLE 4 — Cycle Rejection Twist (Medium/Hard style)

Scenario: exam gives a graph and asks "which edges get REJECTED and why?" rather than just asking for the final MST.

Correct handling: for each rejected edge, you must explicitly state `findParent(u) == findParent(v)` BEFORE that edge was considered — meaning a path already exists between u and v through previously selected edges, so adding this edge would create a redundant cycle. Class notes show this with an 'X' mark next to rejected edges — always justify the X, don't just mark it.

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER

Before you finalize any Kruskal's answer, check:

- Did I sort ALL edges by weight first, before selecting any?
- Did I correctly use Union-Find (not just 'looks connected') to check for cycles?
- Does my final selected-edge count equal n-1 (n = number of nodes)?
- If graph is disconnected, did I explicitly state MST is incomplete?
- For rejected edges, did I justify WHY (same group already) not just mark them rejected?

DIJKSTRA'S ALGORITHM • SIDE 1: CODE

```
#include <iostream>
#include <climits>
using namespace std;

const int MAX = 100;
const int INF = 99999999;

void inputEdgesToAdjMatrix(int adj[MAX][MAX], int n, int e) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = (i == j) ? 0 : INF;
    cout << "Enter edges (Source Destination Weight):\n";
    for (int i = 0; i < e; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        if (u >= 1 && u <= n && v >= 1 && v <= n &&
            u != v && adj[u - 1][v - 1] == INF) {
            adj[u - 1][v - 1] = w;
        } else {
            cout << "Invalid edge. Try again.\n";
            i--;
        }
    }
}

void printAdjMatrix(int adj[MAX][MAX], int n) {
    cout << "\nAdjacency Matrix:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (adj[i][j] == INF) cout << "INF ";
            else cout << adj[i][j] << " ";
        }
        cout << endl;
    }
}

// Dijkstra's Algorithm - single source shortest path, non-negative weights
void dijkstra(int adj[MAX][MAX], int n, int source) {
    int D[MAX]; // shortest distance from source
    bool V[MAX]; // visited flag

    for (int i = 0; i < n; i++) {
        D[i] = INF;
        V[i] = false;
    }
    D[source] = 0;
```

```
while (true) {
    // find minimum distance unvisited node
    int minVal = INF, SN = -1;
    for (int i = 0; i < n; i++) {
        if (!V[i] && D[i] < minVal) {
            minVal = D[i];
            SN = i;
        }
    }
    if (SN == -1) break; // all reachable nodes processed
    V[SN] = true;

    // relax edges from SN
    for (int i = 0; i < n; i++) {
        if (!V[i] && adj[SN][i] != INF &&
            D[SN] + adj[SN][i] < D[i]) {
            D[i] = D[SN] + adj[SN][i];
        }
    }
}

cout << "\nShortest Distance from Node " << source+1 << ":\n";
for (int i = 0; i < n; i++) {
    if (D[i] == INF)
        cout << source+1 << " -> " << i+1 << " = INF\n";
    else
        cout << source+1 << " -> " << i+1 << " = " << D[i] << endl;
}
}

int main() {
    int n, e;
    int adj[MAX][MAX];
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    inputEdgesToAdjMatrix(adj, n, e);
    printAdjMatrix(adj, n);
    int source;
    cout << "\nEnter source node: ";
    cin >> source;
    dijkstra(adj, n, source - 1);
    return 0;
}
```

WHEN TO USE DIJKSTRA'S (Scenario Triggers)

- ▶ Question asks for SHORTEST path from ONE source node to all others (or one target)
- ▶ Graph has WEIGHTED edges, and ALL weights are non-negative
- ▶ "Cheapest route", "minimum cost/time/distance to travel from X to Y" style scenarios
- ▶ GPS/road network, flight cost, network latency minimization scenarios
- ▶ Single starting point given — if question needs shortest path between ALL pairs, use Floyd-Warshall instead

Dijkstra's vs Bellman-Ford vs BFS — Which One?

- ▶ Weighted graph, NO negative edges, single source? → Dijkstra's
- ▶ Weighted graph WITH negative edges (or need cycle detection)? → Bellman-Ford instead
- ▶ UNweighted graph, single source shortest path? → plain BFS is simpler and faster
- ▶ Need shortest path between ALL pairs of nodes? → Floyd-Warshall instead
- ▶ Dijkstra's is essentially 'BFS + always pick the cheapest unvisited node next' instead of FIFO order

MANUAL TRACE METHOD (Selected-Node Table, from Class Notes)

Same table style as Prim's, but tracking DISTANCE FROM SOURCE, not edge weight:

1. Make a table: rows = selected nodes so far, columns = ALL nodes in the graph.
2. Start: source row has $D[\text{source}]=0$, all others = INF (α).
3. Pick the SMALLEST unvisited distance value in the current row → next selected node.
4. New row: copy previous row, UPDATE any column where $D[\text{selected}] + \text{edge_weight} < \text{current value}$.
5. Circle the value you just picked (this locks in the shortest distance to that node).
6. Repeat until all reachable nodes are selected/circled.

Key difference from Prim's table: you ADD cumulative distance ($D[u] + \text{weight}$), not just compare the raw edge weight alone.

COMMON GOTCHAS (Exam Traps)

⚠ Watch out for:

NEGATIVE weights break Dijkstra's — once a node is marked visited, its distance is assumed FINAL. Negative edges can violate this assumption, giving wrong answers silently.

Dijkstra's finds shortest PATH DISTANCE, not minimum edge weight — don't confuse $D[]$ (cumulative) with Prim's $\text{key}[]$ (single edge).

Directed graphs are fine — unlike Prim's/Kruskal's, Dijkstra's does NOT require an undirected graph.

1-indexed input, 0-indexed storage — same off-by-one risk as all other algorithms here.

If a node stays at INF , it's unreachable from the source — state this explicitly if asked.

EXAMPLE 1 — Small Graph (Trace by Hand)

Graph (directed, weighted): $A \rightarrow B=10$, $A \rightarrow C=3$, $C \rightarrow B=1$, $B \rightarrow D=2$, $C \rightarrow D=8$, $D \rightarrow E=7$

Source: A

Step-by-step trace:

- ▶ $D[A]=0$, $D[B]=\text{INF}$, $D[C]=\text{INF}$, $D[D]=\text{INF}$, $D[E]=\text{INF}$
- ▶ Select A ($D=0$) → relax: $D[B]=\min(\text{INF}, 0+10)=10$, $D[C]=\min(\text{INF}, 0+3)=3$
- ▶ Select C ($D=3$, smallest) → relax: $D[B]=\min(10, 3+1)=4$, $D[D]=\min(\text{INF}, 3+8)=11$
- ▶ Select B ($D=4$, smallest) → relax: $D[D]=\min(11, 4+2)=6$
- ▶ Select D ($D=6$, smallest) → relax: $D[E]=\min(\text{INF}, 6+7)=13$
- ▶ Select E ($D=13$) → done, all nodes selected

Shortest distances from A: $A=0$, $B=4$, $C=3$, $D=6$, $E=13$

EXAMPLE 2 — Unreachable Node Scenario

Graph: $A \rightarrow B=5$, $C \rightarrow D=2$ (C, D form a separate component, unreachable from A)

Source: A → $D[A]=0$, $D[B]=5$. $D[C]$ and $D[D]$ remain INF forever (never updated).

Exam expects you to say: "Nodes C and D are unreachable from source A — no directed path exists. Their shortest distance is undefined (INF)."

EXAMPLE 3 — Real-World Scenario (CP-style)

Scenario: "A delivery company wants the cheapest route from their warehouse to every customer location, given road distances between intersections."

How to apply Dijkstra's: warehouse = source node, intersections = nodes, road distances = non-negative weighted edges. Run Dijkstra's from the warehouse — the resulting $D[]$ array directly gives the cheapest route distance to every customer location simultaneously (single-source, multiple destinations).

EXAMPLE 4 — Negative Weight Trap (Medium/Hard style)

Scenario: exam gives a graph WITH a negative edge and asks you to run Dijkstra's on it directly.

Correct handling: DO NOT just run the algorithm blindly. First state: "Dijkstra's algorithm assumes non-negative edge weights. Since this graph contains a negative edge, Dijkstra's may produce an INCORRECT result because once a node is marked visited, we assume its distance is final — a later negative edge could actually offer a shorter path we've already dismissed. Bellman-Ford should be used instead." This reasoning is often worth more marks than blindly computing a (possibly wrong) answer.

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER

Before you finalize any Dijkstra's answer, check:

Did I confirm the graph has NO negative weights before applying Dijkstra's?

Did I pick the SMALLEST cumulative distance at each step, not smallest node index?

Did I relax ALL neighbours of the newly selected node, not just some?

If a node is unreachable, did I explicitly state its distance is INF /undefined?

Did I double check directed vs undirected — did I only set $\text{adj}[u][v]$, matching the graph's actual direction?

BELLMAN-FORD ALGORITHM • SIDE 1: CODE

```
#include <iostream>
using namespace std;

const int MAX = 100;
const int INF = 99999999;

struct Edge {
    int u, v, weight;
};

void inputEdges(Edge edges[], int n, int e) {
    cout << "Enter edges (Source Destination Weight):\n";
    cout << "(Negative weights are allowed)\n";
    for (int i = 0; i < e; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        if (u >= 1 && u <= n && v >= 1 && v <= n && u != v) {
            edges[i].u = u - 1;
            edges[i].v = v - 1;
            edges[i].weight = w;
        } else {
            cout << "Invalid edge. Try again.\n";
            i--;
        }
    }
}

// Bellman-Ford Algorithm - relaxes all edges (n-1) times
void bellmanFord(Edge edges[], int n, int e, int source) {
    int dist[MAX];
    for (int i = 0; i < n; i++) dist[i] = INF;
    dist[source] = 0;

    // relax all edges (n-1) times
    for (int iteration = 1; iteration <= n - 1; iteration++) {
        bool updated = false;
        cout << "\n--- Iteration " << iteration << " ---\n";
        for (int i = 0; i < e; i++) {
            int u = edges[i].u, v = edges[i].v, w = edges[i].weight;
            if (dist[u] != INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                cout << "Edge " << u+1 << " -> " << v+1
                    << " updated. dist[" << v+1 << "] = " << dist[v] << "\n";
                updated = true;
            }
        }
        if (!updated) {
            cout << "No updates. Distances stabilized early.\n";
            break;
        }
    }
}
```

```
// extra pass to detect negative weight cycles
bool hasNegCycle = false;
for (int i = 0; i < e; i++) {
    int u = edges[i].u, v = edges[i].v, w = edges[i].weight;
    if (dist[u] != INF && dist[u] + w < dist[v]) {
        hasNegCycle = true;
        break;
    }
}
if (hasNegCycle) {
    cout << "\nGraph contains a negative weight cycle!\n";
    return;
}
cout << "\nShortest Distance from Node " << source+1 << ":\n";
for (int i = 0; i < n; i++) {
    if (dist[i] == INF)
        cout << source+1 << " -> " << i+1 << " = INF\n";
    else
        cout << source+1 << " -> " << i+1 << " = " << dist[i] << "\n";
}
}

int main() {
    int n, e;
    const int MAXE = MAX * MAX;
    Edge edges[MAXE];
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    inputEdges(edges, n, e);
    int source;
    cout << "\nEnter source node: ";
    cin >> source;
    bellmanFord(edges, n, e, source - 1);
    return 0;
}
```

WHEN TO USE BELLMAN-FORD (Scenario Triggers)

- ▶ Graph has **NEGATIVE** edge weights — Dijkstra's cannot be trusted here
- ▶ Question asks to **DETECT** a negative weight cycle (e.g., arbitrage detection, currency exchange loops)
- ▶ Question mentions "iterations", "relaxation", or gives an iteration-by-iteration table to fill
- ▶ Single-source shortest path needed, but negative edges may exist
- ▶ "Some routes give a refund/bonus (negative cost)" style scenarios

Bellman-Ford vs Dijkstra's — Which One?

- ▶ Negative edges present? → Bellman-Ford (Dijkstra's gives wrong answers)
- ▶ All weights non-negative? → Dijkstra's is faster, prefer it
- ▶ Need to **DETECT** a negative cycle specifically? → Bellman-Ford is the only option here
- ▶ Bellman-Ford is slower ($O(V \times E)$) but more general than Dijkstra's ($O(V^2)$ or better)

MANUAL TRACE METHOD (Iteration Table, from Class Notes)

Use this exact process — matches the class notes' iteration-column table format:

1. Make a table: one column per iteration (1st, 2nd, 3rd... up to $n-1$ iterations).
2. Set $\text{dist}[\text{source}] = 0$, all other nodes = INF (α), **BEFORE** iteration 1.
3. In each iteration, go through **EVERY** edge (in given order) exactly once.
4. For edge (u, v, w) : if $\text{dist}[u] + w < \text{dist}[v]$, **UPDATE** $\text{dist}[v]$ and write the new value in that column.
5. If an edge does **NOT** cause an update, write 'no update' in that column (matches class notes style).
6. Repeat for $(n-1)$ total iterations, **OR** stop early if a full pass causes zero updates.
7. Run **ONE** more pass after the last iteration — if **ANY** edge can still relax, a negative cycle exists.

COMMON GOTCHAS (Exam Traps)

⚠ Watch out for:

Exactly $(n-1)$ iterations are needed in the worst case — this guarantees correctness for graphs with no negative cycle. Stopping too early may miss valid updates.

The **EXTRA** (n -th) pass is what **DETECTS** a negative cycle — if any edge can still relax after $(n-1)$ iterations, a negative cycle exists and shortest paths are undefined.

Edge order matters for the **TRACE** (which values update in which iteration) but **NOT** for the final answer — final shortest distances are the same regardless of edge order.

Unlike Dijkstra's, Bellman-Ford does **NOT** mark nodes 'visited' — it simply relaxes **ALL** edges repeatedly, so it can correctly handle a later negative edge improving an earlier decision.

$\text{dist}[u]$ must **NOT** be INF when relaxing edge (u, v) — check $\text{dist}[u] \neq \text{INF}$ first, otherwise $\text{INF} + \text{weight}$ causes overflow/garbage values.

EXAMPLE 1 — Small Graph with Negative Edge (Trace by Hand)

Graph (directed): $S \rightarrow T=6$, $S \rightarrow Y=7$, $T \rightarrow X=5$, $T \rightarrow Y=8$, $T \rightarrow Z=-4$, $X \rightarrow T=-2$, $Y \rightarrow X=-3$, $Y \rightarrow Z=9$, $Z \rightarrow S=2$, $Z \rightarrow X=7$

Source: S

Step-by-step (simplified trace):

- ▶ Init: $\text{dist}[S]=0$, all others INF
- ▶ Iteration 1: $\text{dist}[T]=6$, $\text{dist}[Y]=7$ (from S). Then $\text{dist}[X]=\min(\text{INF}, 6+5)=11$, $\text{dist}[Z]=\min(\text{INF}, 6-4)=2$
- ▶ Iteration 2: $\text{dist}[X]=\min(11, 7-3)=4$ (via Y better). $\text{dist}[Z]$ stays 2 (no improvement yet)
- ▶ Iteration 3: $\text{dist}[T]=\min(6, 4-2)=2$ (via X better!). This makes $\text{dist}[Z]=\min(2, 2-4)=-2$ (via improved T)
- ▶ Iteration 4: stabilizes, no more updates → stop early

Final shortest distances from S: S=0, T=2, X=4, Y=7, Z=-2 (matches CLRS textbook example, verified against compiled code)

EXAMPLE 2 — Negative Cycle Detection

Graph: $A \rightarrow B=1$, $B \rightarrow C=-3$, $C \rightarrow A=1$ (cycle: $A \rightarrow B \rightarrow C \rightarrow A$ has total weight $1-3+1 = -1$, a **NEGATIVE** cycle)

After $(n-1)=2$ iterations, run **ONE** more pass: edge $B \rightarrow C$ can still relax $\text{dist}[C]$ further.

Exam expects you to say: "Since an edge can still relax after $n-1$ iterations, this graph contains a negative weight cycle. Shortest paths through this cycle are undefined (can be made arbitrarily small by looping)."

EXAMPLE 3 — Real-World Scenario (CP-style)

Scenario: "A currency trader wants to know if there's a sequence of currency exchanges that results in a profit through arbitrage (exploiting exchange rate differences)."

How to apply Bellman-Ford: currencies = nodes, exchange rates = edges. Convert exchange rates to weights using $-\log(\text{rate})$ so that a **PROFITABLE** arbitrage loop becomes a **NEGATIVE** weight cycle. Running Bellman-Ford's negative-cycle-detection step directly answers whether profitable arbitrage exists.

EXAMPLE 4 — Why Not Dijkstra's? (Medium/Hard style)

Scenario: exam gives a graph with one negative edge and asks "Can we use Dijkstra's here? Justify your answer."

Correct handling: "No. Dijkstra's greedily finalizes a node's distance once visited, assuming no future edge can improve it. A negative edge discovered later could still reduce that 'finalized' distance, but Dijkstra's would never revisit it — producing an incorrect result. Bellman-Ford avoids this by relaxing **ALL** edges repeatedly across multiple passes, allowing later negative edges to still correct earlier distances."

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER

Before you finalize any Bellman-Ford answer, check:

- Did I run exactly $(n-1)$ iterations (or stop early only if a full pass had zero updates)?
- Did I run the **EXTRA** pass to check for a negative cycle?
- Did I check $\text{dist}[u] \neq \text{INF}$ before relaxing an edge from u ?
- If a negative cycle exists, did I state shortest paths are undefined (not just report wrong numbers)?
- Did I show the iteration-by-iteration table, not just the final answer?

FLOYD-WARSHALL ALGORITHM • SIDE 1: CODE

```
#include <iostream>
using namespace std;

const int MAX = 100;
const int INF = 99999999;

void inputEdgesToAdjMatrix(int adj[MAX][MAX], int n, int e) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = (i == j) ? 0 : INF;
    cout << "Enter edges (Source Destination Weight):\n";
    cout << "(Negative weights OK, but no negative cycles)\n";
    for (int i = 0; i < e; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        if (u >= 1 && u <= n && v >= 1 && v <= n && u != v) {
            adj[u - 1][v - 1] = w;
        } else {
            cout << "Invalid edge. Try again.\n";
            i--;
        }
    }
}

void printMatrix(int mat[MAX][MAX], int n, string label) {
    cout << "\n" << label << ":\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (mat[i][j] >= INF) cout << "INF\t";
            else cout << mat[i][j] << "\t";
        }
        cout << endl;
    }
}

// Floyd-Warshall - all pairs shortest path
// dist[][] = shortest distance matrix ("D" matrix from class notes)
// next[][] = path reconstruction matrix ("P" matrix from class notes)
void floydWarshall(int adj[MAX][MAX], int n) {
    int dist[MAX][MAX];
    int next[MAX][MAX];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = adj[i][j];
            if (i != j && adj[i][j] != INF) next[i][j] = j;
            else next[i][j] = -1;
        }
    }
}
```

```
// try every node k as an intermediate point
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][k] != INF && dist[k][j] != INF &&
                dist[i][k] + dist[k][j] < dist[i][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
                next[i][j] = next[i][k]; // path goes through k
            }
        }
    }
}

// negative cycle check: dist[i][i] < 0 means a negative cycle
bool hasNegCycle = false;
for (int i = 0; i < n; i++)
    if (dist[i][i] < 0) { hasNegCycle = true; break; }

if (hasNegCycle) {
    cout << "\nGraph contains a negative weight cycle!\n";
    return;
}

printMatrix(dist, n, "Shortest Distance Matrix (D)");

cout << "\nEnter source and destination (1-indexed) for path: ";
int src, dest;
cin >> src >> dest;
src--; dest--;

if (dist[src][dest] >= INF) {
    cout << "No path exists.\n";
    return;
}

cout << "Shortest distance = " << dist[src][dest] << "\n";
cout << "Path: ";
int at = src;
cout << at + 1;
while (at != dest) {
    at = next[at][dest];
    cout << " -> " << at + 1;
}
cout << endl;
}

int main() {
    int n, e;
    int adj[MAX][MAX];
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    inputEdgesToAdjMatrix(adj, n, e);
    floydWarshall(adj, n);
    return 0;
}
```

FLOYD-WARSHALL ALGORITHM • SIDE 2: WHEN, HOW & GOTCHAS

WHEN TO USE FLOYD-WARSHALL (Scenario Triggers)

- ▶ Question asks for shortest path between ALL PAIRS of nodes, not just one source
- ▶ Question gives a matrix of direct costs and asks for the FULL shortest-distance matrix
- ▶ "Find cheapest route between EVERY pair of cities" style scenarios
- ▶ Graph is SMALL (n usually $\leq$ a few hundred) — Floyd-Warshall is $O(n^3)$, too slow for large graphs
- ▶ Question mentions intermediate nodes / "via node k " reasoning
- ▶ Need to detect a negative cycle AND get all-pairs distances in one pass

Floyd-Warshall vs Dijkstra's/Bellman-Ford — Which One?

- ▶ Need ONE source to all others? → Dijkstra's (no negatives) or Bellman-Ford (negatives OK) — both faster
- ▶ Need ALL pairs (every node to every other node)? → Floyd-Warshall
- ▶ Could also run Dijkstra's/Bellman-Ford from EVERY node individually — but Floyd-Warshall is simpler to code (3 nested loops) for small graphs
- ▶ Floyd-Warshall handles negative edges fine (like Bellman-Ford), just not negative cycles

MANUAL TRACE METHOD (Matrix-by-Matrix, from Class Notes)

Use this exact process — matches the class notes' $D_0, D_1, D_2, \dots, D_n$ matrix sequence:

- ▶ 1. Build D_0 = the initial adjacency matrix (INF/ α where no direct edge exists, 0 on diagonal).
- ▶ 2. For each intermediate node k (starting $k=1$), build the NEXT matrix D_k from D_{k-1} .
- ▶ 3. For every cell (i,j) in D_k : check if going THROUGH k is shorter — $D_k[i][j] = \min(D_{k-1}[i][j], D_{k-1}[i][k] + D_{k-1}[k][j])$.
- ▶ 4. Circle/mark any cell that got updated (this shows a shorter path was found via k).
- ▶ 5. Repeat for all $k = 1, 2, \dots, n$. The FINAL matrix D_n is the answer — all-pairs shortest distances.
- ▶ 6. For PATH reconstruction (P matrix): whenever you update $D_k[i][j]$ via k , also update $P[i][j] = k$ (or $P[i][j] = P[i][k]$, depending on convention).

COMMON GOTCHAS (Exam Traps)

⚠ Watch out for:

Loop order MATTERS — k must be the OUTERMOST loop. Putting i or j outermost breaks correctness (you'd be using "not yet fully updated" intermediate values).

Diagonal check for negative cycles — if any $\text{dist}[i][i]$ becomes negative after the full algorithm, a negative cycle exists somewhere in the graph.

The P (path) matrix is a SEPARATE structure from the D (distance) matrix — many students only compute D and forget P is often asked for too.

Self-loops: $\text{dist}[i][i]$ should start at 0, not INF — this is the 'cost of staying at the same node'.

$O(n^3)$ time complexity — if asked to justify why this ISN'T used for huge graphs, cite this cubic blowup directly.

FLOYD-WARSHALL ALGORITHM • SIDE 3: WORKED EXAMPLES

EXAMPLE 1 — Small Graph (Trace by Hand)

Graph (directed, 4 nodes: 1,2,3,4): $1 \rightarrow 3 = -2$, $2 \rightarrow 1 = 4$, $2 \rightarrow 3 = 3$, $3 \rightarrow 4 = 2$, $4 \rightarrow 2 = -1$

Step-by-step (matches classic CLRS 4-node example):

- ▶ D_0 (initial): direct edges only, INF elsewhere, 0 on diagonal
- ▶ After considering $k=1$: paths through node 1 checked — e.g. $2 \rightarrow 1 \rightarrow 3 = 4 + (-2) = 2$, shorter than $2 \rightarrow 3 = 3 \rightarrow$ update $D[2][3] = 2$
- ▶ After considering $k=2$: paths through node 2 checked — $4 \rightarrow 2 \rightarrow 1 = -1 + 4 = 3$, and $4 \rightarrow 2 \rightarrow 3 = -1 + 2 = 1 \rightarrow$ updates ripple
- ▶ After considering $k=3$: paths through node 3 — $1 \rightarrow 3 \rightarrow 4 = -2 + 2 = 0$, shorter routes emerge
- ▶ After considering $k=4$: final matrix D_4 = complete all-pairs shortest distances

Final Answer (verified against compiled code): $\text{dist}[2][3] = 2$, shortest path from 2 to 3 is $2 \rightarrow 1 \rightarrow 3$ (going via node 1 is cheaper than the direct edge $2 \rightarrow 3 = 3$, because $2 \rightarrow 1 = 4$ and $1 \rightarrow 3 = -2$, totaling 2)

Full D matrix (rows/cols = nodes 1,2,3,4): $\text{row1}=[0,-1,-2,0]$, $\text{row2}=[4,0,2,4]$, $\text{row3}=[5,1,0,2]$, $\text{row4}=[3,-1,1,0]$

EXAMPLE 2 — Negative Cycle Detection

Graph: $A \rightarrow B = 1$, $B \rightarrow C = -3$, $C \rightarrow A = 1$ (same cycle from Bellman-Ford example: total weight = -1)

After running Floyd-Warshall fully, check the diagonal: $\text{dist}[A][A]$ will be negative (since $A \rightarrow B \rightarrow C \rightarrow A$ loops back with total -1).

Exam expects you to say: "Since $\text{dist}[A][A] < 0$ after the algorithm completes, a negative weight cycle exists involving node A. All-pairs distances through this cycle are undefined."

EXAMPLE 3 — Real-World Scenario (CP-style)

Scenario: "A logistics company operates between 5 warehouses and wants a complete table of cheapest shipping costs between every pair of warehouses, to answer customer queries instantly without recomputation."

How to apply Floyd-Warshall: warehouses = nodes (small n , so $O(n^3)$ is fine), direct shipping costs = edges.

Running Floyd-Warshall ONCE precomputes the FULL distance matrix — any future "cheapest cost from warehouse X to Y" query becomes an instant $O(1)$ lookup, which is exactly why all-pairs precomputation is valuable here versus running Dijkstra's repeatedly per query.

EXAMPLE 4 — Path Reconstruction Twist (Medium/Hard style)

Scenario: exam gives the final D matrix and P matrix, and asks you to RECONSTRUCT the actual path from node 2 to node 4 (not just the distance).

Correct handling: start at $P[2][4]$ to find the next node on the path from 2. Keep following $P[\text{current}][4]$ until you reach node 4 itself. Class notes show this exact reconstruction: e.g. " $2 \rightarrow 1 \rightarrow 3 \rightarrow 4$ " built by repeatedly consulting the P matrix, not just reading the distance value.

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER

Before you finalize any Floyd-Warshall answer, check:

Is k the OUTERMOST loop in my trace/code (not i or j)?

Did I initialize $\text{dist}[i][i] = 0$ and all non-edges to INF correctly?

Did I check the diagonal for negative cycles after completing all k ?

If asked for a PATH (not just distance), did I use the P matrix, not just D?

Did I justify why Floyd-Warshall ($O(n^3)$) is only used for SMALL graphs, if asked?

FORD-FULKERSON ALGORITHM • SIDE 1: CODE

```
#include <iostream>
using namespace std;

const int MAX = 100;
const int INF = 99999999;

int capacity_[MAX][MAX]; // residual capacity graph
int parent[MAX];

void inputEdgesToAdjMatrix(int n, int e) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            capacity_[i][j] = 0;
    cout << "Enter edges (Source Destination Capacity):\n";
    for (int i = 0; i < e; i++) {
        int u, v, c;
        cin >> u >> v >> c;
        if (u >= 1 && u <= n && v >= 1 && v <= n && u != v && c > 0) {
            capacity_[u - 1][v - 1] += c; // += for parallel edges
        } else {
            cout << "Invalid edge. Try again.\n";
            i--;
        }
    }
}

// Manual BFS to find an augmenting path in the residual graph
bool bfsFindPath(int n, int source, int sink) {
    bool visited[MAX] = {false};
    int queueArr[MAX];
    int qFront = 0, qRear = 0;
    queueArr[qRear++] = source;
    visited[source] = true;
    parent[source] = -1;

    while (qFront < qRear) {
        int u = queueArr[qFront++];
        for (int v = 0; v < n; v++) {
            // residual edge exists if capacity remaining > 0
            if (!visited[v] && capacity_[u][v] > 0) {
                queueArr[qRear++] = v;
                visited[v] = true;
                parent[v] = u;
                if (v == sink) return true;
            }
        }
    }
    return false;
}
```

```
// Ford-Fulkerson (implemented via BFS = Edmonds-Karp variant)
void fordFulkerson(int n, int source, int sink) {
    int maxFlow = 0, pathNumber = 1;

    while (bfsFindPath(n, source, sink)) {
        // find bottleneck capacity along this augmenting path
        int pathFlow = INF;
        int v = sink;
        while (v != source) {
            int u = parent[v];
            if (capacity_[u][v] < pathFlow) pathFlow = capacity_[u][v];
            v = u;
        }
        cout << "\nAugmenting Path " << pathNumber << ": ";
        int pathNodes[MAX], len = 0;
        v = sink;
        while (v != -1) { pathNodes[len++] = v; v = parent[v]; }
        for (int i = len - 1; i >= 0; i--) {
            cout << pathNodes[i] + 1;
            if (i != 0) cout << " -> ";
        }
        cout << " (Bottleneck = " << pathFlow << ")\n";

        // update residual capacities along the path
        v = sink;
        while (v != source) {
            int u = parent[v];
            capacity_[u][v] -= pathFlow; // forward: reduce remaining
            capacity_[v][u] += pathFlow; // backward: allow undo
            v = u;
        }
        maxFlow += pathFlow;
        pathNumber++;
    }
    cout << "\nMaximum Flow = " << maxFlow << endl;
}

int main() {
    int n, e;
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    inputEdgesToAdjMatrix(n, e);
    int source, sink;
    cout << "\nEnter Source node: ";
    cin >> source;
    cout << "Enter Sink node: ";
    cin >> sink;
    fordFulkerson(n, source - 1, sink - 1);
    return 0;
}
```

WHEN TO USE FORD-FULKERSON (Scenario Triggers)

- ▶ Question asks for MAXIMUM FLOW through a network (pipes, roads, data, traffic)
- ▶ Question mentions Source (S), Sink (T), and edge CAPACITIES (not weights/costs)
- ▶ "Maximum amount of water/data/traffic that can flow from A to B" style scenarios
- ▶ Question asks for MINIMUM CUT — the smallest total capacity that, if removed, disconnects source from sink
- ▶ Network has multiple possible paths from source to sink, each with a capacity limit

Key Vocabulary (from Class Notes)

- ▶ Source (S) / Sink (T) — where flow starts / where flow must end up
- ▶ Capacity — maximum flow an edge can carry. Flow — actual amount currently using that edge
- ▶ Augmenting Path — any S-to-T path in the RESIDUAL graph with spare capacity remaining
- ▶ Bottleneck Capacity — the SMALLEST remaining capacity along an augmenting path (limits how much flow that path can add)
- ▶ Residual Graph — shows REMAINING usable capacity on each edge, INCLUDING reverse/backward edges
- ▶ Max-Flow = Min-Cut — this is ALWAYS true (a core theorem); max flow value equals the minimum cut capacity

MANUAL TRACE METHOD (Augmenting Path by Path, from Class Notes)

Use this exact process — matches the class notes' path-listing + bottleneck style:

1. Find ANY path from S to T where every edge still has spare capacity (residual > 0).
2. The BOTTLENECK of this path = the smallest remaining capacity among its edges.
3. Add this bottleneck value to the total flow. Write it as: $S \rightarrow \dots \rightarrow T = [\text{bottleneck value}]$.
4. Update the residual graph: REDUCE forward capacity by bottleneck, INCREASE backward (reverse) capacity by bottleneck.
5. Repeat steps 1-4, finding a NEW augmenting path each time, until NO path from S to T has any remaining capacity.
6. Sum ALL bottleneck values used = Total Maximum Flow.
7. For MIN CUT: after no augmenting path remains, find all nodes STILL reachable from S in the residual graph — call this set 'S-side'. All other nodes = 'T-side'. Min cut = sum of original capacities of edges crossing from S-side to T-side.

COMMON GOTCHAS (Exam Traps)**⚠ Watch out for:**

Backward/reverse edges are ESSENTIAL — they let the algorithm 'undo' a bad earlier flow choice by rerouting. Forgetting to add reverse capacity breaks correctness.

Empty vs non-empty backward edge (from class notes diagram) — a backward edge only allows flow if there's actual forward flow to 'cancel'; $\text{capacity}_{[v][u]}$ starts at 0 and grows as forward flow is pushed.

Full vs non-full forward edge — once $\text{capacity}_{[u][v]}$ reaches 0, that edge can no longer be used in this direction until some flow is undone via the reverse edge.

Order of paths found does NOT affect the final max-flow VALUE, but does affect which specific paths/flows are shown as the 'answer' — both are valid.

Min-Cut nodes are found using BFS/DFS reachability on the FINAL residual graph — this is explicitly noted in class notes as '[BFS]' next to Min Cut.

EXAMPLE 1 — Small Network (Trace by Hand, verified against code)

Graph (directed, capacities): $1 \rightarrow 2 = 16$, $1 \rightarrow 3 = 13$, $2 \rightarrow 3 = 10$, $3 \rightarrow 2 = 4$, $2 \rightarrow 4 = 12$, $4 \rightarrow 3 = 9$, $3 \rightarrow 5 = 14$, $5 \rightarrow 4 = 7$, $4 \rightarrow 6 = 20$, $5 \rightarrow 6 = 4$

Source=1, Sink=6 (classic CLRS max-flow example)

Step-by-step trace:

- ▶ Augmenting Path 1: $1 \rightarrow 2 \rightarrow 4 \rightarrow 6$ (Bottleneck = $\min(16, 12, 20) = 12$)
- ▶ Augmenting Path 2: $1 \rightarrow 3 \rightarrow 5 \rightarrow 6$ (Bottleneck = $\min(13, 14, 4) = 4$)
- ▶ Augmenting Path 3: $1 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6$ (Bottleneck = $\min(9, 10, 7, 8) = 7$)
- ▶ No more augmenting paths remain in the residual graph → STOP

Maximum Flow = $12 + 4 + 7 = 23$ (verified against compiled code output)

EXAMPLE 2 — Finding the Min Cut

Using the SAME network as Example 1, after max flow = 23 is found:

Check which nodes are still reachable from Source(1) in the final residual graph — this defines the S-side of the cut. All remaining nodes form the T-side.

Exam expects you to say: "Sum the ORIGINAL capacities of all edges crossing from S-side to T-side. This sum will equal the max flow (23), confirming the Max-Flow = Min-Cut theorem."

EXAMPLE 3 — Real-World Scenario (CP-style)

Scenario: "A water utility company has a pipe network from a reservoir to a city, with each pipe having a maximum flow capacity (liters/second). What is the maximum water flow achievable to the city?"

How to apply Ford-Fulkerson: reservoir = source, city = sink, pipe junctions = nodes, pipe capacities = edge capacities. Repeatedly find augmenting paths with spare capacity, push the bottleneck amount of flow, until no more augmenting paths exist. The final total = maximum deliverable water flow.

EXAMPLE 4 — Bottleneck Identification Twist (Medium/Hard style)

Scenario: exam gives a specific augmenting path and asks "what is the bottleneck, and which edge causes it?"

Correct handling: identify the SMALLEST remaining (residual) capacity among all edges on that specific path — not the smallest ORIGINAL capacity if some edges already have reduced flow from a previous augmenting path. Always check the CURRENT residual capacity, not the original graph's capacity, since earlier paths may have already used up part of an edge's capacity.

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER**Before you finalize any Ford-Fulkerson answer, check:**

- Did I use CURRENT residual capacities (not original) when finding each new bottleneck?
- Did I correctly update BOTH forward (decrease) and backward (increase) capacities after each path?
- Did I keep finding paths until truly NONE remain (not stop early)?
- If asked for Min Cut, did I use BFS/DFS reachability on the FINAL residual graph?
- Does my Min Cut capacity sum EQUAL my Max Flow value? (They must always match.)